OFFICIAL ENGINEERING PROCESS GUIDE

Project Development & Documentation Guide

How AeroQuant Labs plans, builds, reviews, documents and hands off software — written for every member, today and those who join next.

Version 1.0 | October 2026 | Accra, Ghana | All projects & members

Contents

1. Purpose of the Guide 3
2. Non-Negotiable Rules 4
3. Required Team Roles 6
4. Repository and Documentation Structure 7
5. Standard File Naming 9
6. Project Timeline 10
7. Deliverables 11
D0. Team Charter, Client Intake, Repository Setup and AI Contract 11
D1. Project Proposal, Feasibility and Business Considerations 14
D2. Software Requirements Specification 16
D3. Product Backlog, UX Prototype and Sprint 1 Plan 18
D4. Architecture, CI/CD Baseline and Mid-Project Architecture Review 20
D5. Sprint 1 Review Report and Working Prototype v0.1 22
D6. Sprint 2 Review Report, Test Plan and Quality Evidence 24
D7. Sprint 3 Review Report, Security/Privacy/AI Audit and Observability Evidence 26
D8. Sprint 4 Review Report, Release Candidate and User Manual Draft 28
D9. Final Demo, User Manual, Deployment/Ops Guide and Client Handoff Pack 30
D10. Final Repository Freeze, Post-Mortem, Member Evidence Pack and Peer Review 33
Appendix A. Definition of Ready and Definition of Done 35
Appendix B. Minimum Command and CI Contract 36
Appendix C. AI Usage Log Template 37
Appendix D. Sprint Review Report Template 38
Appendix E. Client Acceptance / Handoff Form Template 39
Appendix F. Red Flags for Weak or Dishonest Project Work 40
Appendix G. Projects Without an External Client (Team Product Projects) 41

Official process guide for every AeroQuant Labs engineering project

Status: Official · Applies to all current and future team members · Applies to every project in this organization · Maintained by: Documentation / Evidence Lead (see §3)

This guide defines **how AeroQuant Labs plans, builds, reviews, documents and hands off software**. It is written for the current team and for every member who joins later: if you follow this guide, your work will be reviewable, traceable and shippable — even to someone who has never met you.

1. Purpose of the Guide

Every AeroQuant Labs project must demonstrate that the team can understand a problem, elicit and validate requirements, plan work, collaborate ethically, design a maintainable solution, implement incrementally, test their work, deploy a usable system, support the client, and explain their own engineering decisions.

AI assistants may be used as productivity tools, but they are not substitutes for understanding. We credit **evidence of engineering practice**, not unexplained generated output. Every submitted artifact must show human judgment, traceability, verification, and individual comprehension.

This guide turns that expectation into a concrete sequence of deliverables (D0–D10), each with required content, a completion checklist and an AI-integrity note.

2. Non-Negotiable Rules

1. **Every project must have genuine end users — and either a genuine client or a named Product Owner.** Two project shapes are valid: **(a) client-sponsored** — an external organization, department, NGO or stakeholder commissions the work; **(b) team product** — the team builds directly for its own end users with no external client. In shape (b), the charter names a Product Owner who plays every client role in this guide, validation evidence must come from real target users, and the team is never its own user (see Appendix G).

2. **Teams must assign clear roles** from §3. Supporting duties may rotate, but each member must own visible work.

3. **The repository is the single source of truth — and every project repository must live under the team's GitHub organization.** No team project may be hosted on a personal account or fork: the DevOps / Release Lead creates each repository inside the AeroQuant Labs organization (or the team monorepo within it) before work begins, and all collaboration happens there. Every deliverable must be pushed to that repository under its `docs/` folder. A document that exists only in chat, email or a personal drive is not submitted.

4. **Every stated deadline is final.** Unless a project plan says otherwise, the internal freeze is **11:59 pm (Accra, GMT)** on the stated day. Work lands as a merged PR before the freeze — "it was pushed at 23:58" does not apply if the PR is still open.

5. **Reports are Markdown in the repository.** When a client-facing or executive copy is required, export a PDF and attach it to the corresponding GitHub Release or share it with the client. Editable sources (Markdown, diagrams, screenshots, PPTX) are always committed.

6. **Slide decks are committed to the repository** (PPTX source) alongside an exported PDF. YouTube demo links must be **unlisted**, accessible, and included both inside the deck and in `videos/youtube_links.md`.

7. **Every deliverable must include**: project name, client name, repository URL, Git commit hash or release tag, team membership and current roles, submission date, and a short AI-assistance disclosure.

8. **Every deliverable must be professional**: readable layout, correct grammar, clear diagrams, consistent headings, numbered pages in exports, named figures/tables, and no raw prompt dumps unless requested in the AI log appendix.

9. **Client data, credentials, private records, proprietary material, passwords, API keys, tokens and identifiable personal information must never be placed in public repositories or pasted into public AI tools.**

10. **A member who cannot explain their submitted work, code, tests, prompt use, design decisions or client requirement loses individual credit for that part of the work.** Repository history is consulted when this is questioned.

3. Required Team Roles

Roles are **generic and permanent** — they belong to the project, never to a person by name in this guide. Any member (current or newly recruited) may hold any role. Your active role assignment for a given project is recorded in `TEAM.md` and in the project charter, and may change between projects.

Role	Core responsibility
Product / Client Manager	Owns client communication, meeting minutes, requirements clarification, client acceptance criteria and deliverable submission coordination. On team product projects, owns the end-user relationship and Product Owner sign-offs instead (Appendix G).
Architecture & Context Lead	Owns architecture decisions, context diagrams, data model, interface contracts, ADRs and technical consistency across the repository.
QA / Validation Lead	Owns test strategy, CI checks, test evidence, bug log, definition of done and release readiness.
DevOps / Release Lead	Owns GitHub setup, branch protection, CI workflows, deployment, secrets handling, release tags and rollback notes.
UX / Research Lead	Owns user observation, personas, workflow mapping, sketches, usability checks and accessibility considerations.
Implementation Leads	Own feature implementation through issues, branches, PRs, tests and documentation. Every project should have **two or more implementers**.
Documentation / Evidence Lead	Owns consistency of deliverable documents, screenshots, repo documentation, changelogs and the final handoff package. This role does not replace each member's obligation to document their own work.

Assignment rules:

- A team of seven may assign one member per role. Smaller or larger teams may combine or split roles, but **no role may be left unowned** and no single member should hold more than two primary roles at once.
- Deputy responsibilities may rotate, but primary ownership must remain visible in the repository (CODEOWNERS / `TEAM.md`) and in meeting minutes.
- When a member leaves or a new member joins, the Product / Client Manager reassigns roles in `TEAM.md` within the same week and records the change in the project minutes.

4. Repository and Documentation Structure

Every project lives in a GitHub repository **owned by the team's GitHub organization** (or a dedicated folder in the organization's monorepo) — never in a member's personal account. The repository is used throughout the engagement for requirements documentation, backlog management, issue tracking, pull requests, code reviews, testing evidence, CI logs, AI usage logs, deployment evidence, release notes, demo links and final handoff materials.

Repository ownership checklist (DevOps / Release Lead, at D0):

- [ ] Repository created **inside the team's GitHub organization**, not a personal account.
- [ ] Organization owns all forks used for the project; no work lives on personal forks.
- [ ] Branch protection and CI enabled on the organization repository.
- [ ] Every member collaborates through the organization (membership invited, no personal clones pushed as the canonical remote).

Every deliverable must land in the repository. A submission pushed nowhere is not a submission.

4.1 Standard repository tree

```
/
├── README.md                  # Project summary, setup, deployment link, team, status
├── TEAM.md                    # Members, GitHub usernames, active role assignments
├── AI_USAGE.md                # Running AI-assistance log (Appendix C format)
├── Makefile or package scripts # setup / lint / test / run / build (Appendix B contract)
├── .github/
│   ├── workflows/             # CI — lint + test on every PR to main
│   ├── PULL_REQUEST_TEMPLATE.md
│   ├── ISSUE_TEMPLATE/
│   └── CODEOWNERS
├── docs/
│   ├── 00_team_charter/
│   ├── 01_proposal/
│   ├── 02_requirements/
│   ├── 03_backlog_ux/
│   ├── 04_architecture_ci/
│   ├── 05_sprint_1/
│   ├── 06_sprint_2_quality/
│   ├── 07_sprint_3_security_observability/
│   ├── 08_sprint_4_release_candidate/
│   ├── 09_final_demo_handoff/
│   └── 10_postmortem_final_freeze/
├── presentations/
│   ├── mid_project_architecture_review/
│   └── final_demo/
├── videos/
│   └── youtube_links.md
├── src/  or  <project-folder>/  # Application code (e.g. aeroquant-labs/)
├── tests/
├── deployment/  or  infra/
└── release-notes/
```

Teams may add framework-specific folders, but the documentation and evidence folders must remain stable — grading, review and handoff all depend on that stability.

4.2 Required GitHub evidence

- Every feature must be tracked by a **GitHub Issue** or equivalent project-board item.
- Every meaningful code change must enter through a **Pull Request** with reviewer comments.
- Every PR must link the relevant issue or requirement and must state **whether AI assistance was used**.
- Every release deliverable must include a **tag** such as `v0.1-prototype`, `v0.2-beta`, `v0.3-staging`, `v1.0-demo` or `v1.0-handoff`.
- The repository history must show **individual contributions spread over time**. A single final bulk upload is not valid engineering evidence.

5. Standard File Naming

Artifact type	Naming pattern	Example
Report (client-facing PDF)	`AQL_<Project>_<DeliverableID>_<ShortName>.pdf`	`AQL_AeroSite_D2_SRS.pdf`
Report source (in repo)	`AQL_<Project>_<DeliverableID>_<ShortName>.md`	`AQL_AeroSite_D2_SRS.md`
Slide deck	`AQL_<Project>_<DeliverableID>_<ShortName>.pptx`	`AQL_AeroSite_D4_ArchitectureReview.pptx`
YouTube link file	`youtube_links.md`	`videos/youtube_links.md`
Release notes	`release-notes/<tag>.md`	`release-notes/v0.2-beta.md`
AI log	`AI_USAGE.md` or `docs/<deliverable>/ai_usage_extract.md`	`AI_USAGE.md`
Client evidence	`client_meeting_<YYYY-MM-DD>.md`	`client_meeting_2026-07-03.md`

6. Project Timeline

Each deliverable (D0–D10) has a fixed place in the sequence. **Concrete calendar dates are set per project at kickoff** and written into the charter (`docs/00_team_charter/`) — this guide defines the cadence, the project plan defines the dates.

ID	Phase	When	Deliverable	Evidence
D0	Kickoff, Week 1	Project day 0	Team Charter, Client Intake, Repository Setup and AI Contract	Source to `docs/00_team_charter` ; repo initialized
D1	Week 2	+1 week	Project Proposal, Feasibility and Business Considerations	Source to `docs/01_proposal` ; client intent evidence
D2	Week 3	+1 week	Software Requirements Specification	Diagrams and source to `docs/02_requirements`
D3	Week 4	+1 week	Product Backlog, UX Prototype and Sprint 1 Plan	Issues/project board live; sketches in `docs/03_backlog_ux`
D4	Week 5	+1 week	Architecture, CI/CD Baseline and Mid-Project Architecture Review	Report + deck in `docs/04_architecture_ci` + `presentations/` ; tag `v0.0-baseline`
D5	Week 6	+1 week	Sprint 1 Review Report and Working Prototype v0.1	`docs/05_sprint_1` ; tag `v0.1-prototype` ; demo evidence
D6	Week 7	+1 week	Sprint 2 Review Report, Test Plan and Quality Evidence	`docs/06_sprint_2_quality` ; tag `v0.2-beta` ; CI/test evidence
D7	Week 8	+1 week	Sprint 3 Review Report, Security/Privacy/AI Audit and Observability Evidence	`docs/07_sprint_3_security_observability` ; tag `v0.3-staging`
D8	Week 9	+1 week	Sprint 4 Review Report, Release Candidate and User Manual Draft	`docs/08_sprint_4_release_candidate` ; tag `v0.9-rc`
D9	Week 10	+1 week	Final Demo, User Manual, Deployment/Ops Guide and Client Handoff Pack	`docs/09_final_demo_handoff` , `presentations/final_demo` , `videos/youtube_links.md` ; tag `v1.0-demo`
D10	Closure	Project end	Final Repository Freeze, Post-Mortem, Member Evidence Pack	`docs/10_postmortem_final_freeze` ; tag `v1.0-handoff`

Slippage is normal; silent slippage is not. If a date will be missed, the Product / Client Manager records the new date and the reason in the project minutes **before** the original deadline.

7. Deliverables

Unless stated otherwise, every deliverable lands as a **merged PR into its `docs/NN_.../` folder** before the deadline; a client-facing PDF export (where required) is attached to the matching GitHub Release or shared with the client by the Product / Client Manager. On team product projects (Appendix G), any item under a deliverable's *Required content and structure* that presupposes a client is satisfied by its Appendix G equivalent: record the substitution in the deliverable instead of leaving the item blank.

D0. Team Charter, Client Intake, Repository Setup and AI Contract

Item	Instruction
Due	Week 1 (project day 0) — date set at kickoff
Lands in	`docs/00_team_charter/` (report + sources + evidence). `README.md`, `TEAM.md` and `AI_USAGE.md` must exist at repo root.

Purpose

This deliverable gets every team operational immediately. It proves that the team exists, has accepted a client/project direction, has joined the repository workflow, has created basic rules for collaboration, and understands the AI-integrity expectations before serious engineering work starts.

Required content and structure

- **A. Team identity**
  - Team/project name, working title and short tagline.
  - Full names, contact channels and GitHub usernames for all members.
  - Professional 100-word biography for each member, written as if introducing the member to a potential client.
  - One team photo or screenshot from the first team meeting.
  - Primary team contact and deputy team contact.
- **B. Team operating agreement**
  - Communication tools and expected response times.
  - Meeting schedule, attendance rules, minute-taking responsibility and decision-recording process.
  - Work division process: how tasks are assigned, reviewed and accepted.

  - Version-control rules: branch names, commit messages, PR size limits, reviewer expectations and merge rules.
  - Conflict-resolution rules and consequences for non-participation, ghosting, dishonesty or late work.
  - Submission process: who prepares the deliverable, who reviews, and what internal freeze time applies before the deadline.
- **C. Client and problem intake**
  - Name of intended client (or shortlist) with contact person and organization.
  - Initial client problem statement in not more than 250 words.
  - Evidence of first contact: email screenshot, message screenshot, meeting minute or call log. Redact private information.
  - Initial list of user groups who may use the system.
  - Fallback plan if the first client is not available by the next deadline.
  - *Team product project (Appendix G): replace client contact evidence with user-research evidence and the Product Owner named in the charter.*
- **D. Repository and platform setup**
  - Repository link; proof every member has access and can commit.
  - Repository initialized with `README.md`, `TEAM.md`, `AI_USAGE.md`, `docs/`, `presentations/`, `videos/`, issues enabled and a project board.
  - Team YouTube channel name and URL for unlisted videos.
  - Submission coordinator identified (Product / Client Manager).
- **E. AI contract**
  - Signed statement by all members acknowledging that AI may assist but cannot replace understanding.
  - Agreement to record AI use in `AI_USAGE.md`.
  - Agreement not to paste confidential client data, credentials or private records into public AI tools.
  - Agreement that any member unable to explain their own work may receive zero individual credit for that work.

Quality checklist before submission

- [ ] The repo link opens successfully for every member and any external reviewer.
- [ ] Every member's GitHub username is correct.
- [ ] The team has at least one viable client direction.
- [ ] The team agreement includes real consequences and is not generic filler.
- [ ] The AI contract is explicit and named by all members.

AI integrity note

- AI-generated team agreements are easy to detect because they are vague. The agreement must include concrete meeting times, tools, response-time rules and consequences that fit your team.
- AI may help polish biographies or wording, but every member must approve their own biography and AI usage must be disclosed.

D1. Project Proposal, Feasibility and Business Considerations

Item	Instruction
Due	Week 2
Lands in	`docs/01_proposal/` (report + source + client evidence + diagrams). Update `README.md` with the project summary and client-facing problem statement.

Purpose

This deliverable converts the early client conversation into a realistic software project proposal. It proves that the team understands the client, the problem, the intended users, the scope, the risks, the business/legal context and the delivery plan.

Required content and structure

- **A. Executive summary**
  - One-page summary of the project title, client, problem, users, proposed solution and expected value.
  - Clear statement of the task to be undertaken in this engagement.
  - Statement of what will **not** be built in this engagement to prevent uncontrolled scope growth.
- **B. Client, users and intended use**
  - Client organization or person, contact person and role.
  - User groups, their current workflow and their pain points.
  - How the client intends to use the system after the engagement.
  - Minimum three-year relevance argument: why this solution could remain useful beyond the engagement.
- **C. Overall functionality and preliminary requirements**
  - High-level feature list grouped by user role.
  - Preliminary functional requirements.
  - Preliminary non-functional requirements: usability, reliability, security, privacy, performance, maintainability and accessibility.
  - Major assumptions and constraints.
- **D. Main system components**
  - Decompose the system into logical or architectural components.

  - Explain why the decomposition is appropriate.
  - Include an initial context diagram or block diagram.
  - Identify major data objects and external systems, if any.
- **E. Process and milestone plan**
  - Chosen development process: agile, iterative refinement, phased delivery or justified hybrid.
  - Sprint plan mapping to this guide's D5–D8 sprint reviews.
  - Milestones that match this guide's deliverables.
  - Visibility plan: how the team will keep the client and team members informed.
- **F. Business, legal and ethical considerations**
  - Copyright/license arrangement: transfer, non-exclusive license or other agreed terms for client use.
  - Use of open-source libraries and expected license obligations.
  - Data protection and confidentiality issues.
  - Whether any NDA, trade-secret, patent, conflict-of-interest or proprietary-data issue may arise.
  - No-warranty engagement disclaimer for the client.
- **G. Risk analysis**
  - Top ten project risks: probability, impact, owner and mitigation.
  - Client availability risk and fallback plan.
  - Technical risk and fallback plan.
  - Team participation risk and escalation process.
  - AI misuse risk and verification plan.

Quality checklist before submission

- [ ] The client and users are real, not imaginary.
- [ ] Scope is achievable within the planned engagement window.
- [ ] Risks include both technical and human risks.
- [ ] Business considerations are not ignored.
- [ ] The proposal is specific enough to become an SRS next week.

AI integrity note

- Do not use AI to invent client needs. Every claim about client workflow must be traceable to an interview, observation, message or document.
- AI may help organize risks or polish wording, but the final content must match the actual client conversation.

D2. Software Requirements Specification

Item	Instruction
Due	Week 3
Lands in	`docs/02_requirements/` (report + editable source + diagrams + client validation evidence). Convert validated stories into GitHub Issues after submission.

Purpose

This deliverable establishes the system requirements baseline. It proves that the team can move from a vague client need to precise, testable, traceable requirements and user stories.

Required content and structure

- **A. Requirements overview**
  - Purpose, scope and system boundary.
  - Stakeholder map and user roles.
  - High-level narrative of the functional requirements.
  - Assumptions, constraints and dependencies.
- **B. Use case model**
  - One complete UML use case diagram showing the overall system functionality.
  - Natural-language description for each use case.
  - Identification of primary and secondary actors.
  - Preconditions, main success scenario, alternatives/exceptions and postconditions for major use cases.
- **C. User stories**
  - A single numbered list of user stories in the format: *As a ‹role›, I want ‹goal›, so that ‹benefit›*.
  - Acceptance criteria for each story using clear Given/When/Then or checklist format.
  - Preconditions and postconditions linked to story numbers.
  - Initial identification of epics that must be decomposed in D3.
  - Traceability from each use case to one or more user stories.
- **D. Non-functional requirements**
  - Usability requirements.
  - Performance and availability expectations.

- Security, privacy and access-control requirements.
- Maintainability, portability and deployment constraints.
- Accessibility and inclusive-use considerations.
- Data backup, audit and logging needs where relevant.

- **E. Data and external interface requirements**
  - Key data entities and fields at a conceptual level.
  - External systems, APIs, files, payment systems, email/SMS services or authentication providers.
  - Data sensitivity classification: public, internal, confidential or personal.
  - Initial data retention and deletion expectations.
- **F. Glossary and validation evidence**
  - Glossary of domain terms and special meanings.
  - Client validation notes: what was confirmed, what changed, what is still unknown.
  - Open questions that must be resolved in D3 or D4.

Quality checklist before submission

- [ ] Every functional requirement is understandable and testable.
- [ ] Every user story has a real user role and benefit.
- [ ] No duplicate or conflicting stories remain unresolved.
- [ ] Non-functional requirements are measurable where possible.
- [ ] The client has seen or validated the requirements baseline.

AI integrity note

- AI may suggest requirement categories, but it cannot validate client needs. Client validation evidence is mandatory.
- Any requirement that cannot be explained by at least one team member and linked to a stakeholder conversation may be rejected.

D3. Product Backlog, UX Prototype and Sprint 1 Plan

Item	Instruction
Due	Week 4
Lands in	`docs/03_backlog_ux/`. Every story must exist as a GitHub Issue or linked project-board item.

Purpose

This deliverable transforms the SRS into a prioritized, estimated, sprint-ready backlog and a first user-interface direction. It forces the team to plan the work realistically before code generation accelerates the wrong solution.

Required content and structure

- **A. Refined product backlog**
  - Break down epics into smaller user stories and show which epics produced which stories.
  - Use Fibonacci story points from 1, 2, 3, 5 and 8. Avoid stories larger than 8; decompose them.
  - Assign priority: Must / Should / Could / Won't this engagement (or High/Medium/Low).
  - Include acceptance criteria, story owner, dependencies and risk notes.
  - Calculate total backlog size and planned sprint capacity.
- **B. Backlog traceability**
  - Map each user story to SRS requirement numbers and use case numbers.
  - Identify stories deferred beyond the engagement scope.
  - Identify stories requiring client clarification before implementation.
  - Flag stories involving sensitive data or security decisions.
- **C. UX sketches and prototype evidence**
  - Low-fidelity sketches, wireframes or simple clickable mockups for core screens.
  - At least three main user flows.
  - Usability assumptions and accessibility considerations.
  - Evidence that at least one intended user or client representative reviewed the direction.
- **D. Sprint 1 plan**
  - Select Sprint 1 stories totaling a realistic portion of backlog capacity.
  - State the Sprint 1 goal and expected prototype functionality.
  - Assign owners for every selected story.

  - Define tests to be written for Sprint 1 stories before or alongside implementation.
  - State Definition of Ready and Definition of Done (Appendix A).

- **E. GitHub project board setup**
  - Board columns: Backlog / Ready / In Progress / Review / Done, with labels such as `requirement`, `frontend`, `backend`, `database`, `testing`, `docs`, `bug`, `security`, `ai-assisted`.
  - Each issue has acceptance criteria and an owner.
  - Each issue is linked to a milestone or sprint.
  - The Product / Client Manager keeps the board current weekly.

Quality checklist before submission

- [ ] No story is larger than 8 points unless explicitly justified.
- [ ] Every Sprint 1 story has an owner, acceptance criteria and planned tests.
- [ ] The backlog reflects client priorities, not only developer preferences.
- [ ] UX sketches reflect the actual workflow.
- [ ] GitHub Issues and the submitted report agree.

AI integrity note

- AI may help split epics or suggest acceptance criteria, but the team must review every story for real client value.
- Do not accept AI-proposed UI screens that do not match the user's actual workflow.

D4. Architecture, CI/CD Baseline and Mid-Project Architecture Review

Item	Instruction
Due	Week 5
Lands in	`docs/04_architecture_ci/` + `presentations/mid_project_architecture_review/` (report, PPTX + PDF deck). Create release tag `v0.0-baseline`.

Purpose

This deliverable proves that the team can design before rushing into code generation. It establishes the architecture, technology stack, repository quality gates, deployment approach and mid-project review evidence.

Required content and structure

- **A. Architecture report**
  - System context diagram showing users, external systems and boundaries.
  - Container/component diagram showing major frontend, backend, data, service and deployment components.
  - Data model: ERD, schema draft or domain model appropriate to the chosen stack.
  - API/interface contract: endpoint list, request/response summaries or interface definitions.
  - Architecture Decision Records for at least three important decisions: stack choice, database choice, authentication choice, deployment choice or major architectural pattern.
  - Security and privacy design assumptions.
- **B. Repository engineering baseline**
  - Command contract with at least `setup`, `lint`, `test` and `run` / `build` mapped to the chosen stack (Makefile or npm scripts — see Appendix B).
  - GitHub Actions workflow that runs setup, lint and test on pull requests to `main`.
  - Pull request template requiring issue link, tests, screenshots for UI work and AI-assistance disclosure.
  - Issue templates for feature, bug and documentation tasks.
  - CODEOWNERS or equivalent reviewer protection for CI workflow, dependency files and architecture documents.
  - Branch protection: `main` must not receive direct unreviewed feature commits.
- **C. Deployment and environment plan**

  - Chosen deployment platform and rationale.
  - Environment variables and secrets plan **without exposing real secrets**.
  - Local setup instructions that a new developer can follow.
  - Staging and final-release plan.
  - Backup and rollback expectations if the system stores client data.
- **D. Mid-project architecture review deck/video**
  - Maximum 10 minutes if video is required; every team member must speak meaningfully.
  - Slides must cover problem, client, users, requirements, backlog, architecture, repo/CI evidence, risks, current progress and lessons learned.
  - Include the repository link, project board link, release tag and video link inside the deck.
  - Audio and visuals must be clear. Use landscape slides with readable text.
- **E. Baseline release**
  - Create GitHub release/tag `v0.0-baseline`.
  - Release notes must state what exists, what is scaffolded, what tests run and what is not yet implemented.
  - CI must pass, or any failure must be explained with a remediation plan approved by the client or engagement lead.

Quality checklist before submission

- [ ] Architecture diagrams are consistent with requirements and backlog.
- [ ] A fresh developer can run — or understand how to run — the project.
- [ ] CI actually executes commands, not placeholder echo statements.
- [ ] The deck demonstrates engineering decisions, not marketing language alone.
- [ ] Every member can defend their role and contribution.

AI integrity note

- AI may generate draft diagrams or boilerplate CI files, but the team must verify that commands run for the actual stack.
- Architecture decisions must be justified using project constraints. Do not submit fashionable stack choices without rationale.

D5. Sprint 1 Review Report and Working Prototype v0.1

Item	Instruction
Due	Week 6
Lands in	`docs/05_sprint_1/`. Create release tag `v0.1-prototype` with release notes.

Purpose

This deliverable proves that the team can build the first working slice of the system using disciplined workflow. The prototype must show real functionality, even if incomplete. A static mockup alone is not enough unless the project is explicitly design-tool focused and approved.

Required content and structure

- **A. Sprint summary** — sprint goal and dates covered; updated sprint backlog with planned, completed, carried-over and cancelled items; summary of implemented functionality; links to the relevant GitHub Issues, Pull Requests and release tag.
- **B. Requirements and backlog changes** — stories changed, split, added or removed during the sprint and why; whether new stories went to the backlog or were implemented immediately, with justification; updated traceability impact.
- **C. Engineering evidence** — PRs merged during the sprint and reviewers involved; tests added or updated; CI status links; deployment link or demo clip; known defects and technical debt.
- **D. Metrics and reflection** — story points planned vs completed; velocity; burndown or progress chart; lessons learned and next-sprint changes; contribution summary by member.
- **E. Client feedback** — evidence of client/user review, message, meeting or demo; what was accepted, rejected or changed; new questions for the next client meeting. *Team product project (Appendix G): the end users are the client — record their feedback gathered via WhatsApp, Google Forms / Jotform or phone calls, plus Product Owner review.*
- **F. Minimum prototype expectations**
  - At least one core user flow partially works end-to-end or through a clearly explained mocked dependency.
  - Basic project setup, linting and tests run through CI.
  - User-visible changes are documented with screenshots.
  - Deployment or local-run instructions are updated.

Quality checklist before submission

- [ ] There is a release tag `v0.1-prototype`.

- [ ] The prototype can be demonstrated from repo instructions or a deployment link.
- [ ] The sprint report includes velocity, burndown and lessons learned.
- [ ] There is clear evidence of PR review and testing.
- [ ] Client feedback is captured, even if brief.

AI integrity note

- AI-generated code must be reviewed, tested and explained. A PR with AI code but no tests or explanation is not acceptable.
- The team should be able to point to at least one defect caught by human review or testing.

D6. Sprint 2 Review Report, Test Plan and Quality Evidence

Item	Instruction
Due	Week 7
Lands in	`docs/06_sprint_2_quality/` + `tests/`. Create release tag `v0.2-beta`.

Purpose

This deliverable shifts the team from "it works on my machine" to verifiable quality. It continues sprint reporting while requiring a formal test plan and evidence that quality gates are part of the engineering process.

Required content and structure

- **A–E.** Sprint summary, requirements/backlog changes, engineering evidence, metrics and reflection, client feedback — same structure as D5.
- **F. Test plan**
  - Testing scope and levels: unit, integration, API, UI, end-to-end, usability or manual tests as appropriate.
  - Test cases mapped to requirements/user stories.
  - Business rules that must not break.
  - Tools used for testing and how to run the tests.
  - Minimum test coverage target for business logic, and rationale if coverage cannot be measured automatically.
- **G. Quality evidence**
  - CI result summary for the sprint.
  - Test execution report with pass/fail status.
  - Bug log: defect, severity, owner, status and link to issue/PR.
  - Code review findings and how they were resolved.
  - Refactoring or maintainability improvements made during the sprint.

Quality checklist before submission

- [ ] The project has a written test plan, not only ad hoc screenshots.
- [ ] Tests can be run through the documented command contract (Appendix B).
- [ ] The sprint release is tagged `v0.2-beta`.

- [ ] Bug log and known issues are honest and specific.
- [ ] Quality evidence is linked to actual CI/PR/test artifacts.

AI integrity note

- AI may suggest tests, but tests must assert the actual behavior of our system. No meaningless tests that only increase coverage numbers.
- Every member should be ready to explain at least one test, one bug and one review comment.

D7. Sprint 3 Review Report, Security/Privacy/AI Audit and Observability Evidence

Item	Instruction
Due	Week 8
Lands in	`docs/07_sprint_3_security_observability/` . `AI_USAGE.md` must be updated. Create release tag `v0.3-staging` .

Purpose

This deliverable adds responsible engineering controls. It requires teams to examine how their system handles data, access, failure, logging, AI use and production-like visibility before the release-candidate stage.

Required content and structure

- **A–E.** Sprint summary, requirements/backlog changes, engineering evidence, metrics and reflection, client feedback — same structure as D5.
- **F. Security and privacy audit**
  - Threat model for at least five realistic threats relevant to the project.
  - Authentication and authorization plan, if applicable.
  - Input validation, file upload, payment, messaging or email risk controls, if applicable.
  - Secrets management plan: no keys/tokens/passwords in the repository.
  - Dependency risk: how libraries/packages are selected and updated.
  - Data privacy table: data collected, purpose, sensitivity, storage location, retention and who can access it.
- **G. Responsible AI audit**
  - Extract from `AI_USAGE.md` summarizing tools used, type of assistance and verification steps.
  - List of AI-generated or AI-assisted PRs and how the team verified them.
  - Statement confirming that no confidential client data was pasted into public AI tools.
  - Risks introduced by AI assistance and mitigations.
- **H. Observability evidence**
  - Logging plan: what events/errors are logged and what must not be logged.
  - Error tracking or monitoring screenshot/link where feasible, or local logging evidence for small projects.

- How the team will know the app has failed after deployment.
- Known reliability limitations and fallback plan.

Quality checklist before submission

- [ ] No secrets are exposed in the repository.
- [ ] Security risks are project-specific, not generic copy-paste.
- [ ] The AI usage log is honest and traceable to PRs or deliverables.
- [ ] A staging release or clear deployable build exists.
- [ ] The team can explain how failures will be detected.

AI integrity note

- Any undisclosed AI-assisted code discovered during review may be treated as an integrity breach.
- AI may support threat brainstorming, but the final audit must reflect the real architecture and data flows.

D8. Sprint 4 Review Report, Release Candidate and User Manual Draft

Item	Instruction
Due	Week 9
Lands in	`docs/08_sprint_4_release_candidate/` + draft manuals. Create release tag `v0.9-rc`.

Purpose

This deliverable moves the product into release-candidate state. It ensures that most functionality is complete, known defects are visible, the client can try the system, and user documentation is ready before the final demo week.

Required content and structure

- **A–E.** Sprint summary, requirements/backlog changes, engineering evidence, metrics and reflection, client feedback — same structure as D5.
- **F. Release candidate checklist**
  - Feature completion matrix: planned, completed, partially completed, deferred and removed features.
  - Known issues with severity and planned resolution or accepted limitation.
  - Deployment link or installation package/instructions.
  - Regression test evidence after final major changes.
  - Client/user acceptance testing plan for the final week.
- **G. User manual draft**
  - Audience and prerequisites.
  - Installation/access instructions.
  - Main features explained in user-friendly language.
  - Walkthrough of the main scenario with screenshots.
  - Walkthroughs for at least two additional scenarios.
  - Troubleshooting section and support contacts for the handoff period.
- **H. Operations/deployment draft**
  - How to deploy or redeploy the system.
  - Environment variables and secrets list using **placeholders, never real values**.
  - Database migration/seed instructions if applicable.

- Backup, rollback and maintenance notes.
- Admin account setup or role management if applicable.

Quality checklist before submission

- [ ] The release candidate is tagged `v0.9-rc`.
- [ ] The client can access or be shown the system before the final demo.
- [ ] The user manual has screenshots and real workflows.
- [ ] Known issues are not hidden.
- [ ] The team has a plan for the final demo (D9) and closure (D10).

AI integrity note

- AI can help improve user manual wording, but screenshots and workflows must come from the actual system.
- Do not let AI hallucinate features that the product does not have.

D9. Final Demo, User Manual, Deployment/Ops Guide and Client Handoff Pack

Item	Instruction
Due	Week 10
Lands in	`docs/09_final_demo_handoff/`, `presentations/final_demo/`, `videos/youtube_links.md`. Final deck as PPTX + PDF, manuals as PDF exports shared with the client. Create release tag `v1.0-demo`.

Purpose

This is the main end-of-project demonstration. The team must show a working solution, explain the engineering process, hand over documentation and prove that the client can understand what has been delivered.

Required content and structure

- **A. Final demo video**
  - Unlisted YouTube video, 5–10 minutes, clear audio and readable screen recording or camera footage.
  - Must pitch the problem, client, users, solution and value.
  - Must demonstrate the main user flow and at least two additional meaningful flows.
  - Must mention known limitations honestly.
  - Every team member must appear or speak meaningfully unless an accommodation has been approved.
- **B. Final demo slide deck**
  - Problem and client context.
  - Requirements and scope summary.
  - Architecture and technology stack.
  - Sprint progress and major changes.
  - Testing, CI/CD, security, privacy and AI-use evidence.
  - Live/deployed system link or installation instructions.
  - Client feedback and acceptance status.
  - Repository link, project board link, release tag and video link.
- **C. Final user manual**

  - Installation/access instructions and user prerequisites.
  - Main features explained clearly for non-developers.
  - Main scenario walkthrough with screenshots.
  - At least two additional workflows with screenshots.
  - Troubleshooting, FAQ and support handoff information.
  - Accessibility notes where relevant.
- **D. Deployment and operations guide**
  - Full deployment procedure.
  - Build commands, environment variables and infrastructure notes.
  - Database migration and seed instructions.
  - Backup, recovery and rollback notes.
  - Admin/maintenance tasks.
  - Post-engagement maintenance options for the client.
- **E. Client handoff pack**
  - Client acceptance draft or signed acceptance if available.
  - Final requirements-to-delivered-features traceability table.
  - Known limitations and recommended next steps.
  - License/copyright arrangement or engagement-use disclaimer.
  - No-warranty statement unless a different arrangement has been approved.
- **F. Individual review readiness**
  - Each member must be ready to explain their contribution, at least one requirement, one design decision, one code area, one test, one PR review and one lesson learned.
  - Members must not rely on a team spokesperson to answer everything.
  - Repository history and PRs will be used during questioning.

Quality checklist before submission

- [ ] The YouTube link is unlisted but accessible.
- [ ] The slide deck is readable and professional.
- [ ] The demo shows the actual product, not only slides.
- [ ] The user manual and deployment guide are sufficient for handoff.
- [ ] The system has a tagged `v1.0-demo` release.
- [ ] Every member is prepared for individual questioning.

AI integrity note

- Do not use AI-generated narration to hide weak team understanding. Every member may be questioned on the demo.
- Any AI-generated images, voiceovers or scripts must be disclosed, and the engineering content must still be explainable by the team.

D10. Final Repository Freeze, Post-Mortem, Member Evidence Pack and Peer Review

Item	Instruction
Due	Closure week — project end
Lands in	`docs/10_postmortem_final_freeze/`. Create release tag `v1.0-handoff` before the deadline.

Purpose

This is the final closure task. It captures corrections after the final demo, freezes the repository, records client acceptance, documents lessons learned and gives each member a record of individual contribution and interview readiness.

Required content and structure

- **A. Final repository freeze**
  - Final release tag `v1.0-handoff` with release notes.
  - Final commit hash recorded in the closure report.
  - `README.md` updated with project summary, setup instructions, deployment link, team members and client/handoff status.
  - All deliverable folders complete and consistently named.
  - No exposed secrets, test credentials or private client data in the repo.
- **B. Post-mortem report**
  - What went well and why.
  - What failed or underperformed and why.
  - Root-cause analysis for major schedule, quality, client or team problems.
  - What the team would do differently in a real client engagement.
  - Final state of requirements: delivered, partially delivered, deferred and cancelled.
  - Maintenance recommendations for the client.
- **C. Client acceptance and handoff evidence**
  - Signed client acceptance, email acceptance or meeting minutes recording the client's response.
  - Client training or walkthrough evidence if conducted.
  - List of accounts, access steps or deployment ownership notes shared with the client. **Do not include passwords in the document.**

  - Final license/copyright/usage understanding.
- **D. Final AI usage audit**
  - Final `AI_USAGE.md` summary.
  - List of AI-assisted deliverables or PRs.
  - Human verification evidence: tests, reviews, refactoring and manual checks.
  - Statement of any AI limitations or hallucinations encountered.
- **E. Member evidence pack**
  - One page per member summarizing role, commits, PRs, reviewed PRs, issues owned, documents contributed, tests written and client/team contributions.
  - Each member answers five interview-style prompts: what problem did you solve; what did you design; what did you implement; how did you test; what would you improve.
  - Each member identifies one code file, one test file, one requirement and one design decision they can defend in an interview.
  - Each member reflects on responsible AI use and what they learned without AI.
- **F. Peer review**
  - Members confidentially evaluate contribution, reliability, communication, technical work, review quality and integrity of the team they worked with.
  - Results feed the post-mortem and future role assignments. Non-participation is recorded by the Product / Client Manager.

Quality checklist before submission

- [ ] The final repo tag `v1.0-handoff` exists before the deadline.
- [ ] The closure report names the final commit hash and repo link.
- [ ] Client acceptance or client feedback is included.
- [ ] Individual evidence is specific and verifiable through GitHub.
- [ ] AI usage has been logged honestly.
- [ ] Peer review is completed individually.

AI integrity note

- Individual reflections must be written in the member's own voice. AI-polished text is less important than concrete evidence and interview readiness.
- A member who cannot discuss their individual evidence page may lose individual credit for that work.

Appendix A. Definition of Ready and Definition of Done

Definition of Ready

- The story has a clear user role, goal and benefit.
- Acceptance criteria are written and understandable.
- Dependencies and preconditions are known.
- The story is small enough to complete in a sprint.
- The team knows how it will test the story.
- The Product / Client Manager has confirmed the story's priority or marked it as a hypothesis.

Definition of Done

- Code is committed through a branch and pull request — no direct unreviewed `main` work.
- At least one non-author reviews the PR and leaves a meaningful comment.
- CI passes, or an approved exception is documented.
- Tests are added or updated where appropriate.
- Screenshots or demo evidence are attached for visible UI changes.
- Documentation, release notes or user manual are updated if behavior changes.
- AI assistance is disclosed in the PR and `AI_USAGE.md` when used.
- The issue is linked and moved to Done only after acceptance criteria are met.

Appendix B. Minimum Command and CI Contract

Every repository must provide a stack-specific implementation of a common command contract. The exact commands depend on the chosen stack, but the **names must remain stable** for review and CI. Implement as a `Makefile`, npm scripts, or both.

```
setup:   # install dependencies, e.g. npm ci / pip install -r requirements.txt
lint:    # run static checks, e.g. eslint . / ruff check .
test:    # run unit/integration/E2E tests, e.g. npm test / pytest
run:     # start the app locally or print clear instructions
build:   # create production build where applicable
```

- By **D4**, `setup`, `lint` and `test` must exist and must not be empty placeholders.
- By **D6**, tests should cover the core business logic, or the team must explain why automated coverage is technically unsuitable.
- By **D8**, build/deployment instructions must be reliable enough for a new developer to follow.
- The team must not weaken CI checks late in a project to hide broken code.

Appendix C. AI Usage Log Template

Field	Required entry
Date	When the AI tool was used.
Tool	ChatGPT, Claude, Gemini, Copilot, Cursor, Windsurf, etc.
Task	What the team wanted help with.
Prompt summary	Short summary — not the full confidential prompt.
Output used	What was copied, adapted or rejected.
Human changes	What the team changed after AI output.
Verification	Tests, review, client check, documentation check or manual validation.
Related artifact	PR number, issue number, document section or deliverable ID.
Owner	Member responsible for verifying the AI-assisted work.

Appendix D. Sprint Review Report Template

- Title page: project name, team, sprint number, dates, repo URL, release tag and final commit hash.
- Sprint goal.
- Sprint backlog table: story ID, points, owner, planned/completed status and PR link.
- Functionality completed and how to access/demo it.
- Screenshots, video snippets or deployment link.
- User stories changed, split, added or removed.
- Tests and CI evidence.
- Velocity: planned points, completed points, carry-over points and interpretation.
- Burndown/progress chart.
- Client feedback.
- Lessons learned and next-sprint changes.
- AI usage summary for the sprint.
- Contribution summary by member.

Appendix E. Client Acceptance / Handoff Form Template

- Project name and team name.
- Client organization/person and contact.
- Summary of delivered system.
- Deployment/access link or installation medium.
- Features accepted.
- Known limitations or deferred items.
- Documentation received: user manual, deployment guide, release notes.
- Usage/license understanding.
- No-warranty statement unless otherwise agreed.
- Client name, role, signature/date or email confirmation.

Appendix F. Red Flags for Weak or Dishonest Project Work

- A single large commit appears immediately before the deadline.
- Most code is written by one member while others only edit documents.
- The app works only in a demo video but cannot be run from repository instructions.
- `AI_USAGE.md` is empty while code/document style suggests extensive AI assistance.
- Requirements do not match the client's actual workflow.
- No tests, no code review comments and no meaningful issues exist despite a "completed" app.
- Diagrams, manual and UI screenshots describe features that do not exist.
- Members cannot explain code they supposedly authored.
- Client feedback is missing or suspiciously generic.
- Sensitive data or credentials are committed to the repo.

Appendix G. Projects Without an External Client (Team Product Projects)

Some projects are built by the team directly **for end users** — public tools, internal systems, open products — with no sponsoring client. These projects follow this guide exactly, with one substitution: wherever the guide says *client*, **the end users are the client**. The client is not a single entity but the collective user base: all client-side information and evidence is taken from real users directly, through **WhatsApp chats, Google Forms / Jotform surveys, or phone calls**, and recorded as dated evidence files in `docs/` (affirmed under *Affirmation* below).

- **The team is never its own user.** Dogfooding is welcome for development, but it does not count as validation. Evidence must come from people outside the team who match the target audience.
- **A Product Owner is named in the charter (D0).** This is the internal owner who accepts scope, prioritizes the backlog and signs off launch — playing every client role across D0–D10. Where practical, the Product Owner should not be the implementer of the work being accepted.
- **The user base is the client — evidence comes from many users, not one client entity.** Interviews, surveys, analytics, usability sessions and beta feedback are collected from real end users **via WhatsApp chats, Google Forms / Jotform surveys, or phone calls**, and filed as dated evidence files in `docs/`, exactly like client meeting minutes: who was reached, when, through which channel, and what they said.

Client-based wording in this guide	Team product project equivalent
D0 — client intake and first-contact evidence	Problem statement backed by user research: target-audience interviews, market/competitor notes (users reached via WhatsApp, Google Forms / Jotform or phone), analytics from similar tools
D1 — client intent evidence and three-year relevance	Product hypothesis, target users, success metrics (KPIs) and why the product stays useful beyond the engagement
D2 — client validation of the requirements baseline	User validation: at least two or three sessions with target users per major flow; open-questions log
D3 — backlog reflects client priorities; user reviews UX direction	Product Owner prioritization against KPIs; usability test of the prototype with at least one target user
D5–D8 — "Client feedback" in every sprint review	"User feedback" section: test sessions, beta cohort responses and analytics gathered via WhatsApp, Google Forms / Jotform or phone calls — what was accepted, rejected or changed
D8 — client acceptance testing plan	Beta / early-access plan naming non-team testers

D9 — client handoff pack and acceptance	Launch sign-off by the Product Owner, runbook, and named ongoing ownership (who monitors and supports the product)
D10 — client acceptance evidence	Launch review: KPI vs hypothesis, user-feedback summary, and a go / iterate / retire decision
Appendix E — client acceptance form	Product acceptance sign-off: same fields, with the Product Owner as signatory and "features accepted" as launch scope

Affirmation: items that cannot be met literally on team product projects.

Some entries under each deliverable's *Required content and structure* are written from the client's side of the engagement: signed client approvals, client meeting minutes, client feedback records, client acceptance forms. On a team product project **those items cannot be submitted — and their absence is not a defect.** Reviewers must judge them as follows:

- **There is no single client entity — so a meeting with "the client" is never expected.** Every requirement that presumes talking to *the client* is fulfilled from the users themselves: information and evidence are gathered directly from end users **via WhatsApp chats, Google Forms / Jotform surveys, or phone calls**, and their replies and submissions are filed in `docs/` as the record — who was reached, when, through which channel, what was decided. Never invent one authoritative client person or entity, and never fabricate a single client's minutes or sign-off (Appendix F). Product Owner reviews complement, but do not replace, user voices.
- The item is **fully met** when the equivalent from the table above is present and dated in `docs/`. No apology, permission or special note is required; write "per Appendix G" beside the entry.
- Where no equivalent exists, write an explicit line in the deliverable: `N/A – team product project (Appendix G): <why it does not apply> – <substitute evidence, or "none applies" with reasoning>`. An explicit N/A with reasoning is complete evidence; a silently blank item is not.
- **Never fabricate client evidence.** Invented client names, fake meeting minutes or mocked sign-offs are dishonest project work (Appendix F) regardless of intent.
- Deadlines, review cadence, quality bars and the meaning of every other requirement stay identical. Only the *source* of evidence changes: from the client to real end users and the Product Owner.

Everything else — roles, repository structure, deliverables, timeline, AI-integrity rules and Appendices A–F — applies unchanged.

Colophon. Source: `docs/PROJECT_DEVELOPMENT_GUIDE.md` · Typeset with github-markdown-css (MIT) and Paged.js (MIT) · Typeface: Manrope (SIL Open Font License). Generated 2026-11-01 · © AeroQuant Labs.